VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

CSE REPORT
CLASS CC01 – LAB 2
SUBJECT:
SOC

Instructor: Phạm Kiều Nhật Anh

Student	ID
Hoàng Đình Hiếu	2310945

I. CARRY LOOK-AHEAD ADDER (CLA)

Source code:

```
`timescale 1ns / 1ps

/**
 * @param a first 1-bit input
 * @param b second 1-bit input
 * @param g whether a and b generate a carry
 * @param p whether a and b would propagate an incoming carry
 */
module gp1(input wire a, b,
        output wire g, p);
  assign g = a & b;
  assign p = a ^ b;
endmodule

/**
 * Computes aggregate generate/propagate signals over a 4-bit window.
 * @param gin incoming generate signals
 * @param pin incoming propagate signals
 * @param cin the incoming carry
 * @param gout whether these 4 bits internally would generate a carry-out (independent of cin)
 * @param pout whether these 4 bits internally would propagate an incoming carry from cin
 * @param cout the carry outs for the low-order 3 bits
 */
module gp4(input wire [3:0] gin, pin,
        input wire cin,
        output wire gout, pout,
        output wire [2:0] cout);
assign cout[0]=gin[0] | (pin[0]&cin);
assign cout[1]=gin[1] | (pin[1]&gin[0]) | (pin[1]&pin[0]&cin);
assign cout[2]=gin[2] | (pin[2]&gin[1]) | (pin[2]&pin[1]&gin[0]) | (pin[2]&pin[1]&pin[0]&cin);
assign gout=gin[3] | (pin[3]&gin[2]) | (pin[3]&pin[2]&gin[1]) | (pin[3]&pin[2]&pin[1]&gin[0]);
assign pout=pin[3]&pin[2]&pin[1]&pin[0];
  // TODO: your code here

endmodule

/** Same as gp4 but for an 8-bit window instead */
module gp8(input wire [7:0] gin, pin,
        input wire cin,
        output wire gout, pout,
        output wire [6:0] cout);
assign cout[0]=gin[0] | (pin[0]&cin);
assign cout[1]=gin[1] | (pin[1]&gin[0]) | (pin[1]&pin[0]&cin);
assign cout[2]=gin[2] | (pin[2]&gin[1]) | (pin[2]&pin[1]&gin[0]) | (pin[2]&pin[1]&pin[0]&cin);
assign cout[3] = gin[3] | (pin[3]&gin[2]) | (pin[3]&pin[2]&gin[1]) | (pin[3]&pin[2]&pin[1]&gin[0]) |
(pin[3]&pin[2]&pin[1]&pin[0]&cin);
   assign cout[4] = gin[4] | (pin[4]&gin[3]) | (pin[4]&pin[3]&gin[2]) | (pin[4]&pin[3]&pin[2]&gin[1])
            | (pin[4]&pin[3]&pin[2]&pin[1]&gin[0]) | (pin[4]&pin[3]&pin[2]&pin[1]&pin[0]&cin);
   assign cout[5] = gin[5] | (pin[5]&gin[4]) | (pin[5]&pin[4]&gin[3]) | (pin[5]&pin[4]&pin[3]&gin[2])
            | (pin[5]&pin[4]&pin[3]&pin[2]&gin[1]) | (pin[5]&pin[4]&pin[3]&pin[2]&pin[1]&gin[0])
            | (pin[5]&pin[4]&pin[3]&pin[2]&pin[1]&pin[0]&cin);
   assign cout[6] = gin[6] | (pin[6]&gin[5]) | (pin[6]&pin[5]&gin[4]) | (pin[6]&pin[5]&pin[4]&gin[3])
            | (pin[6]&pin[5]&pin[4]&pin[3]&gin[2]) | (pin[6]&pin[5]&pin[4]&pin[3]&pin[2]&gin[1])
            | (pin[6]&pin[5]&pin[4]&pin[3]&pin[2]&pin[1]&gin[0]) |
(pin[6]&pin[5]&pin[4]&pin[3]&pin[2]&pin[1]&pin[0]&cin);
```

```
assign gout=gin[7]
            | (pin[7]&gin[6])
            | (pin[7]&pin[6]&gin[5])
            | (pin[7]&pin[6]&pin[5]&gin[4])
            | (pin[7]&pin[6]&pin[5]&pin[4]&gin[3])
            | (pin[7]&pin[6]&pin[5]&pin[4]&pin[3]&gin[2])
            | (pin[7]&pin[6]&pin[5]&pin[4]&pin[3]&pin[2]&gin[1])
            | (pin[7]&pin[6]&pin[5]&pin[4]&pin[3]&pin[2]&pin[1]&gin[0]);
assign pout=pin[7]&pin[6]&pin[5]&pin[4]&pin[3]&pin[2]&pin[1]&pin[0];
  // TODO: your code here

endmodule

module cla
  (input wire [31:0]  a, b,
   input wire        cin,
   output wire [31:0] sum);
wire [31:0]g;
wire [31:0]p;
wire [7:0]gg;
wire [7:0]pp;
wire [30:0] c;
wire [6:0]cout8;
genvar i;
generate
for(i=0;i<32;i=i+1)begin
gp1 gp1a(
.a(a[i]),
.b(b[i]),
.g(g[i]),
.p(p[i])
);
end
endgenerate
gp4 gp4_1(.gin(g[3:0]),.pin(p[3:0]),.cin(cin),.gout(gg[0]),.pout(pp[0]),.cout(c[2:0]));

gp4 gp4_2(.gin(g[7:4]),.pin(p[7:4]),.cin(cout8[0]),.gout(gg[1]),.pout(pp[1]),.cout(c[6:4]));

gp4 gp4_3(.gin(g[11:8]),.pin(p[11:8]),.cin(cout8[1]),.gout(gg[2]),.pout(pp[2]),.cout(c[10:8]));

gp4 gp4_4(.gin(g[15:12]),.pin(p[15:12]),.cin(cout8[2]),.gout(gg[3]),.pout(pp[3]),.cout(c[14:12]));

gp4 gp4_5(.gin(g[19:16]),.pin(p[19:16]),.cin(cout8[3]),.gout(gg[4]),.pout(pp[4]),.cout(c[18:16]));

gp4 gp4_6(.gin(g[23:20]),.pin(p[23:20]),.cin(cout8[4]),.gout(gg[5]),.pout(pp[5]),.cout(c[22:20]));

gp4 gp4_7(.gin(g[27:24]),.pin(p[27:24]),.cin(cout8[5]),.gout(gg[6]),.pout(pp[6]),.cout(c[26:24]));

gp4 gp4_8(.gin(g[31:28]),.pin(p[31:28]),.cin(cout8[6]),.gout(gg[7]),.pout(pp[7]),.cout(c[30:28]));

gp8 gp8a(.gin(gg[7:0]),.pin(pp[7:0]),.cin(cin),.gout(),.pout(),.cout(cout8[6:0]));
  assign sum[3:0]    = p[3:0]    ^ {c[2:0], cin};
  assign sum[7:4]    = p[7:4]    ^ {c[6:4], cout8[0]};
  assign sum[11:8]   = p[11:8]   ^ {c[10:8], cout8[1]};
  assign sum[15:12]  = p[15:12]  ^ {c[14:12], cout8[2]};
  assign sum[19:16]  = p[19:16]  ^ {c[18:16], cout8[3]};
  assign sum[23:20]  = p[23:20]  ^ {c[22:20], cout8[4]};
```

```
 assign sum[27:24]  = p[27:24]  ^ {c[26:24], cout8[5]};
 assign sum[31:28]  = p[31:28]  ^ {c[30:28], cout8[6]};

endmodule
```

Code explanation:
***gp1:**
The gp1 module is a 1-bit Generate–Propagate block used in a Carry Lookahead Adder (CLA).
It processes one bit of the inputs a and b and produces:
-g → Generate signal (this bit generates a carry)
-p → Propagate signal (this bit propagates an incoming carry)

assign g = a & b;
A carry is generated if both a and b are 1
Truth table:

a	b	g
0	0	0
0	1	0
1	0	0
1	1	1

assign p = a ^ b;
A carry is propagated if exactly one of a or b is 1
Truth table:

a	b	p
0	0	0
0	1	1
1	0	1
1	1	0

***gp4:**
1. Purpose of gp4
The gp4 module is a 4-bit Generate/Propagate block used in a Carry Lookahead Adder (CLA).
Its job is to:
-Compute the carry outputs for 4 bits in parallel
-Determine whether this 4-bit block:
+Generates a carry on its own (gout)
+Propagates an incoming carry through all 4 bits (pout)
This allows the adder to be much faster than a ripple-carry adder, because carries are computed using logic instead of waiting sequentially.
2. Inputs and Outputs
input wire [3:0] gin, pin; // Generate & propagate for each bit
input wire cin; // Carry input to the 4-bit block
output wire gout, pout; // Group generate & propagate
output wire [2:0] cout; // Carry outputs for bits 0–2

Meaning of inputs:
-gin[i] = 1 → bit i generates a carry
-pin[i] = 1 → bit i propagates a carry
-cin → carry coming into this 4-bit block
Meaning of outputs:
-cout[0] → carry into bit 1
-cout[1] → carry into bit 2
-cout[2] → carry into bit 3
-gout → this 4-bit block generates a carry regardless of cin
-pout → this 4-bit block propagates cin through all 4 bits
3. Carry Equations Explained
✅ **Carry into bit 1 (cout[0])**

assign cout[0] = gin[0] | (pin[0] & cin);

Meaning:
A carry into bit 1 happens when:
-Bit 0 generates a carry (gin[0]), OR
-Bit 0 propagates the input carry (pin[0] & cin)

This is the standard CLA carry formula:
C1 = G0 + P0·C0
✅ **Carry into bit 2 (cout[1])**
assign cout[1] = gin[1]
| (pin[1] & gin[0])
| (pin[1] & pin[0] & cin);

Meaning:
A carry into bit 2 happens when:
-Bit 1 generates a carry
-Bit 1 propagates and bit 0 generates
-Bit 1 & bit 0 propagate and cin is 1

Formula:
C2 = G1 + P1·G0 + P1·P0·C0
✅ **Carry into bit 3 (cout[2])**
assign cout[2] = gin[2]
| (pin[2] & gin[1])
| (pin[2] & pin[1] & gin[0])
| (pin[2] & pin[1] & pin[0] & cin);

Meaning:
A carry into bit 3 happens if:
-Bit 2 generates
-Bit 2 propagates and bit 1 generates
-Bits 2 & 1 propagate and bit 0 generates
-Bits 2, 1, 0 all propagate and cin is 1

Formula:
C3 = G2 + P2·G1 + P2·P1·G0 + P2·P1·P0·C0
4. Group Generate (gout)
assign gout = gin[3]
| (pin[3] & gin[2])
| (pin[3] & pin[2] & gin[1])
| (pin[3] & pin[2] & pin[1] & gin[0]);

This tells whether this entire 4-bit block generates a carry regardless of cin.
It becomes 1 if:
-Bit 3 generates a carry
-OR bit 3 propagates and bit 2 generates
-OR bits 3 & 2 propagate and bit 1 generates
-OR bits 3, 2, 1 propagate and bit 0 generates
5. Group Propagate (pout)
assign pout = pin[3] & pin[2] & pin[1] & pin[0];

This is 1 only if all 4 bits propagate a carry.
Meaning:
If cin = 1 and pout = 1, then the carry will pass through all 4 bits unchanged.
***gp8:**
The algorithm is the same as gp4 but extended.

***cla:**

1. Module purpose

module cla (input [31:0] a, b, input cin, output [31:0] sum);

This module adds two 32-bit numbers a and b with an input carry cin and produces a 32-bit sum using carry lookahead logic (fast adder).

2. Internal signals

wire [31:0] g, p; // bitwise generate & propagate
wire [7:0] gg, pp; // block (4-bit) generate & propagate
wire [30:0] c; // internal carries
wire [6:0] cout8; // carries between 8-bit blocks

-g[i] = generate for bit i
-p[i] = propagate for bit i
-gg[k], pp[k] = generate/propagate for each 4-bit block
-c = carries inside 4-bit blocks
-cout8 = carries between 4-bit blocks (calculated by gp8)

3. Bitwise generate/propagate (gp1 blocks)

for(i=0;i<32;i=i+1)
* gp1 gp1a(.a(a[i]), .b(b[i]), .g(g[i]), .p(p[i]));*

For each bit:
-g[i] = a[i] & b[i]
-p[i] = a[i] ^ b[i]
This prepares data for fast carry computation.

4. 4-bit carry lookahead blocks (gp4)

Example:

gp4 gp4_1(.gin(g[3:0]), .pin(p[3:0]), .cin(cin),
* .gout(gg[0]), .pout(pp[0]), .cout(c[2:0]));*

Each gp4:
-Computes 3 internal carries
-Produces block generate (gg[k])
-Produces block propagate (pp[k])
-Each block handles 4 bits
There are 8 blocks × 4 bits = 32 bits.

5. 8-block carry lookahead (gp8)

gp8 gp8a(.gin(gg[7:0]), .pin(pp[7:0]),
* .cin(cin), .cout(cout8[6:0]));*

This computes the carry going into each 4-bit block, so all block carries are calculated in parallel → very fast.

6. Sum calculation

Example:

assign sum[3:0] = p[3:0] ^ {c[2:0], cin};

Sum formula:
sum=p ⊕ carry

Each 4-bit group uses:
-Its internal carries (c)
-Or block carry (cout8)
This is done for all 32 bits.

Source code:

```
`include "cla2.0.v"
module SystemDemo(
```

```
  input wire [3:0] btn,
  output wire [5:0] led
);
  wire [31:0] sum;
  cla cla_inst(
    .a(32'd26),
   .b({28'b0, btn[3], btn[2], btn[1], btn[0]}),
   .cin(1'b0), .sum(sum)
  );
  assign led = sum[5:0];
endmodule
```

Code explanation:

1. include "cla2.0.v"

`include "cla2.0.v"

This line includes the Verilog file cla2.0.v, which presumably contains the definition of the cla module. Without this, the compiler wouldn't know what cla is.

2. Module Declaration

module SystemDemo(
* input wire [3:0] btn,*
* output wire [5:0] led*
);

-Module name: SystemDemo.
-Inputs: btn[3:0] → 4 push buttons (could be from a development board like Basys or Nexys).
-Outputs: led[5:0] → 6 LEDs to show the result of some computation.

3. Wire Declaration

wire [31:0] sum;

Declares a 32-bit wire sum to hold the output of the adder.

4. CLA Instantiation

cla cla_inst(
* .a(32'd26),*
* .b({28'b0, btn[3], btn[2], btn[1], btn[0]}),*
* .cin(1'b0),*
* .sum(sum)*
);

This creates an instance of your CLA adder called cla_inst.
Ports are connected as follows:

Port	Connection	Explanation
a	32'd26	Constant decimal 26 as a 32-bit input.
b	{28'b0, btn[3], btn[2], btn[1], btn[0]} → creates a 32-bit number.	Concatenates 28 zeros with 4-bit button value
cin	1'b0	Carry-in is 0.
sum	sum	32-bit output wire to capture the sum.

Essentially, this adder computes:
sum=26+btn_value
where btn_value is the 4-bit number represented by your buttons.

5. Assign Output to LEDs

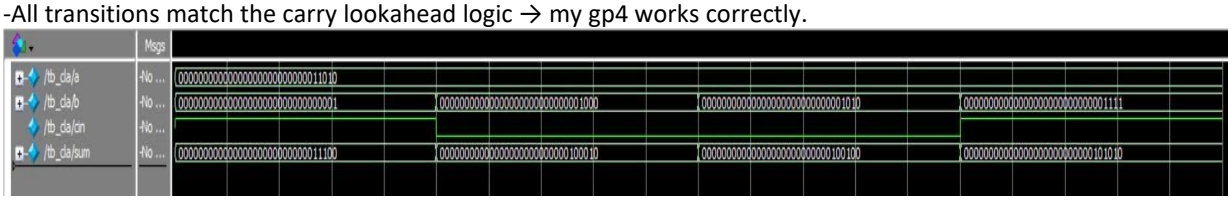
assign led = sum[5:0];

-Assigns the lowest 6 bits of the 32-bit sum to the LEDs.
-Why 6 bits? Because the sum can go up to:
26+15=41
-41 in binary is 101001, which fits in 6 bits. Perfect for led[5:0].

Result:

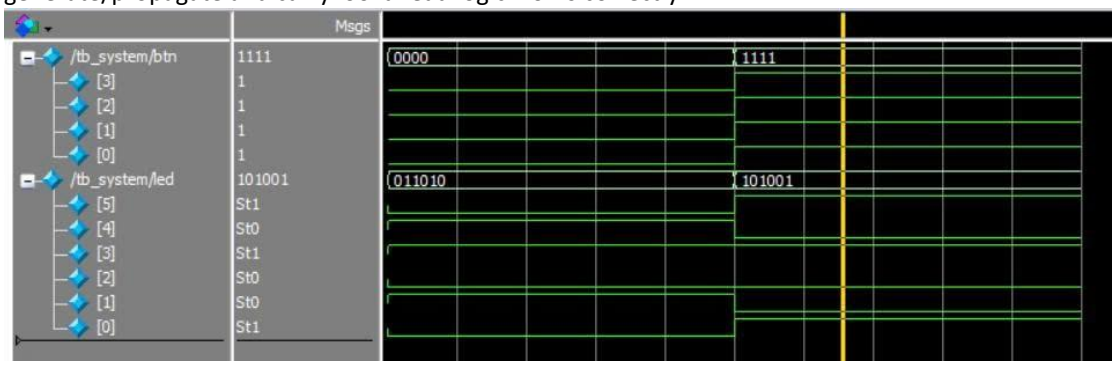
WAVEFORM EXPLANATION:

-gin and pin change over time → they control carry generation and propagation.
-cout[2:0] shows the carries computed instantly from gin, pin, and cin (here cin = 0).
-gout = 1 only when the 4-bit block generates a carry by itself.
-pout = 1 only when all pin = 1, meaning the whole block can propagate a carry.
-All transitions match the carry lookahead logic → my gp4 works correctly.



WAVEFORM EXPLANATION:

-a[31:0] and b[31:0]: These are the two 32-bit inputs. You see them changing at each test step to apply different addition cases.
-cin: Input carry. When it is 0, the adder computes a + b. When it is 1, it computes a + b + 1.
-sum[31:0]: Output result of the addition. After each change of a, b, or cin, sum updates immediately because the CLA is purely combinational (no clock delay).
-Each transition in sum matches the binary addition of the current a, b, and cin, proving that the generate/propagate and carry lookahead logic works correctly.



WAVEFORM EXPLANATION:

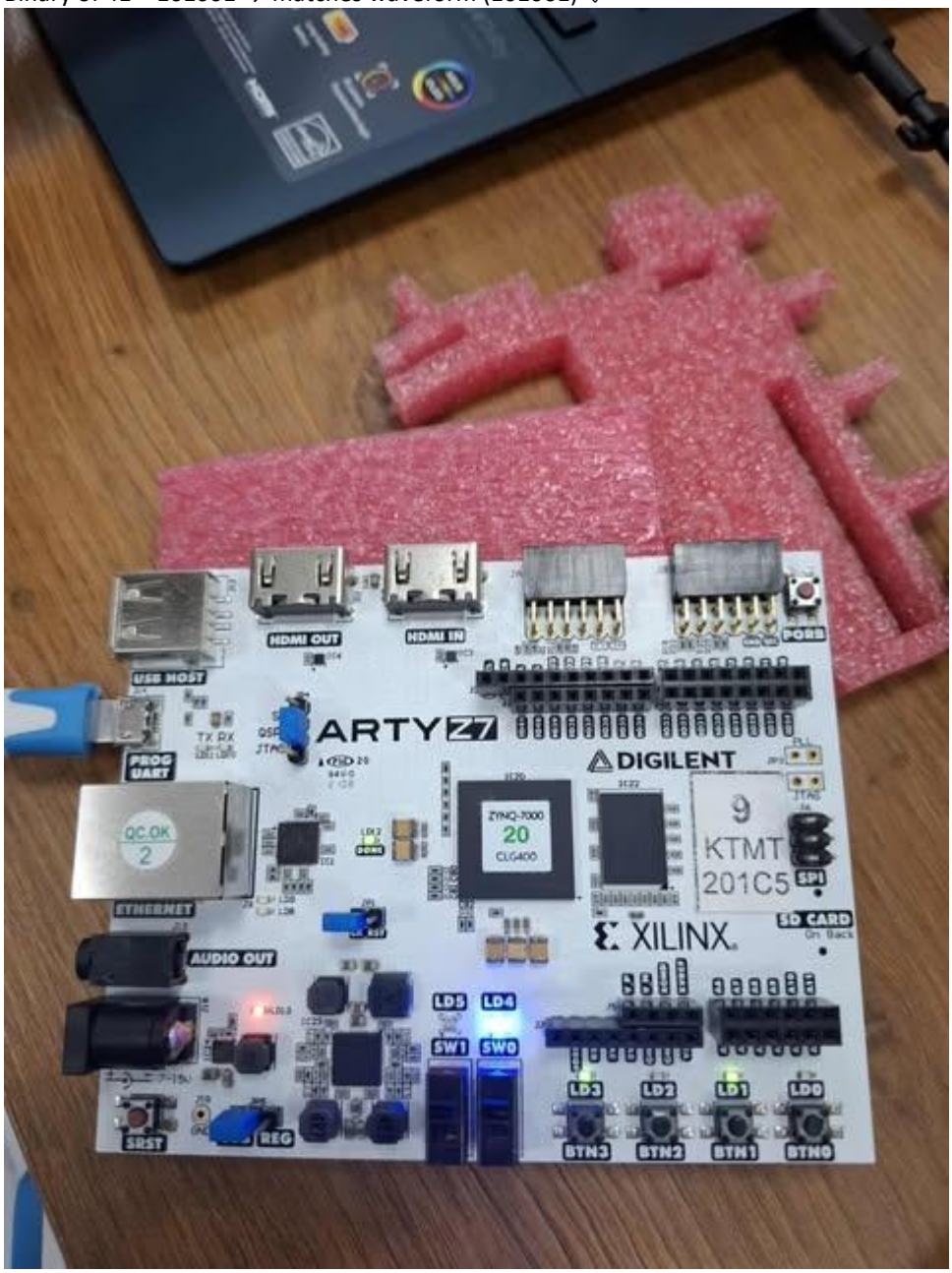
1. Signals in the Waveform

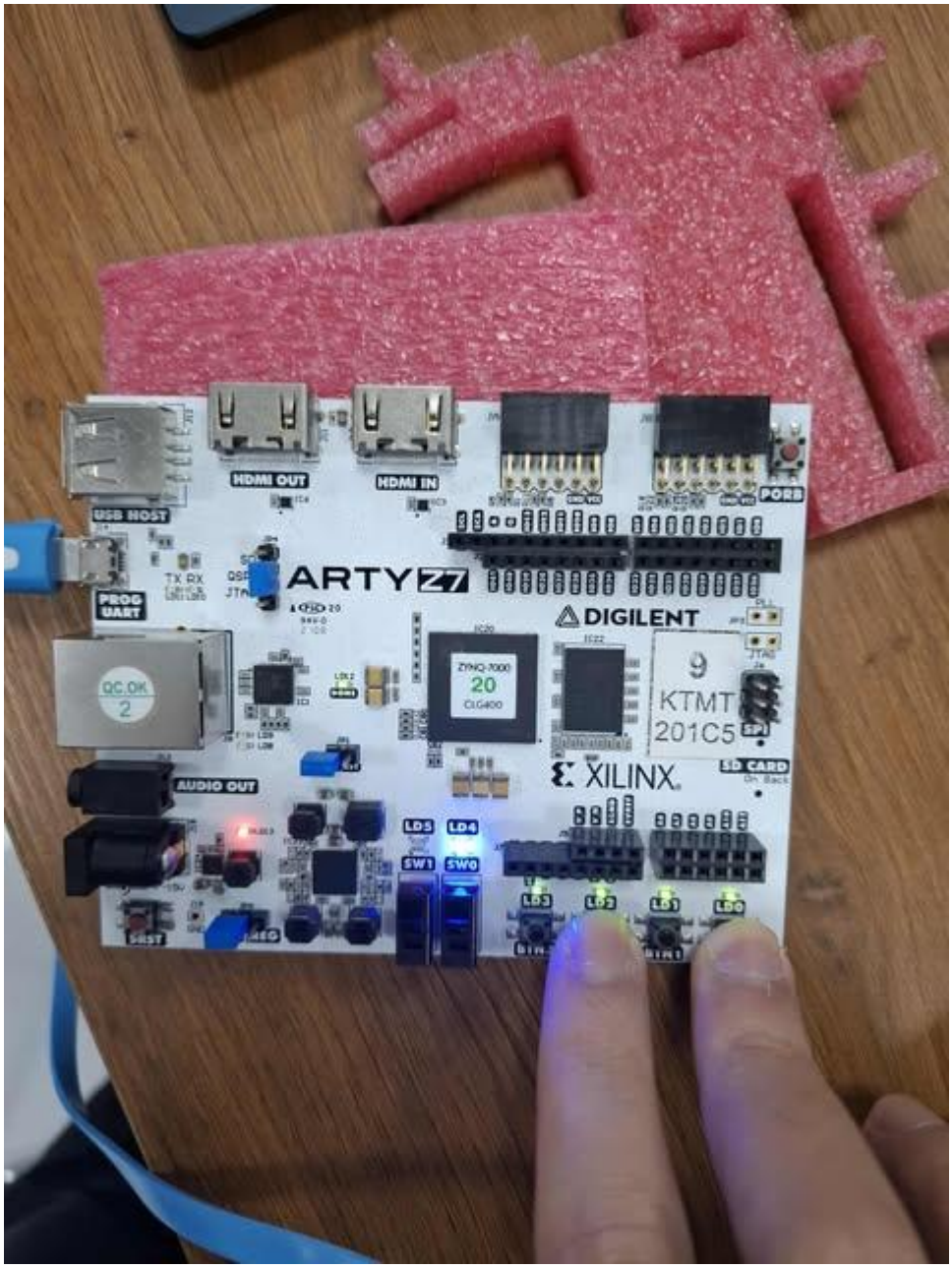
/tb_system/btn[3:0] → 4-bit button input.
-Shows 0000 initially, then changes to 1111.
-This matches the 4-bit input from your buttons in your SystemDemo module.
/tb_system/led[5:0] → 6-bit LED output.
-Shows 011010 when btn=0000.
-Shows 101001 when btn=1111.

2. Check the LED values

My module does:
sum=26+btn_value
Case 1: btn = 0000 (decimal 0)
26+0=26
Binary of 26 = 011010 → matches waveform (011010) ✅

Case 2: btn = 1111 (decimal 15)
26+15=41
Binary of 41 = 101001 → matches waveform (101001) ✅





II. RISC_V ASSEMBLY

Source code:

```
        .section .data
input_string:   .asciz "Hello world!"

        # tohost/fromhost are data locations used by Spike
        .align 8
tohost: .byte 0
        .align 8
fromhost: .byte 0


        .section .text
        .globl _start
_start:
```

```
    # Load the address of the input string into a0
    la a0, input_string
    # Your code here!
    li t1, 97
    li t2, 122
loop:
  lb t0, 0(a0)
  beqz t0, end_program
  blt t0, t1, next_char
  bgt t0, t2, next_char
  addi t0, t0, -32
  sb t0, 0(a0)

next_char:
  addi a0, a0, 1
  j loop

end_program:
    # jump here when you're done
    #j end_program
    li a1, 10
    ecall
```

Code explanation:

✅ **Purpose of the Program**

The code converts all lowercase letters ('a'–'z') in the string "Hello world!" to uppercase in-place, then exits.

So "Hello world!" becomes "HELLO WORLD!".

1. Data Section

.section .data

input_string: .asciz "Hello world!"

-Creates a null-terminated string (*0*) stored in memory.

-The label *input_string* holds its starting address.

tohost and *fromhost* are for the Spike simulator and not relevant to your logic.

2. Text Section Setup

.section .text

.globl _start

_start:

Defines the program entry point (*_start*).

3. Load string address

la a0, input_string

-Loads the address of the first character of the string into register *a0*.

-*a0* acts like a walking pointer to each character.

4. Preload useful constants

li t1, 97 # ASCII 'a'

li t2, 122 # ASCII 'z'

These values define the ASCII lowercase range [97..122].

5. Loop Through Characters

loop:

* lb t0, 0(a0) # load byte at *a0 into t0*

* beqz t0, end_program*

-Load the current character.

-If it's 0 (\0 terminator), exit the loop.

6. Check if character is lowercase
blt t0, t1, next_char # if t0 < 'a', skip
bgt t0, t2, next_char # if t0 > 'z', skip

If character is outside 'a'..'z', do nothing.
7. Convert lowercase → uppercase
addi t0, t0, -32
sb t0, 0(a0)

ASCII rule:
'a' = 97
'A' = 65
97 - 32 = 65
So subtracting 32 converts lowercase to uppercase.
8. Move to next character
next_char:
  addi a0, a0, 1
  j loop

-Increment the pointer.
-Loop again.
9. Program Exit
end_program:
  li a1, 10
  ecall

Calls an exit syscall (depends on your environment).
Result:

Address	Binary	Value
0x00200000 - 0x00200003	input_string 48 45 4C 4C	H, E, L, L
0x00200004 - 0x00200007	4F 20 57 4F	O, , W, O
0x00200008 - 0x0020000B	52 4C 44 21	R, L, D, !
0x0020000C - 0x0020000F	00 00 00 00	0

My program converts every lowercase a–z in "Hello world!" into uppercase by subtracting 32 from the ASCII value.

Memory shows:

48 45 4C 4C 4F → HELLO
20 → space
57 4F 52 4C 44 → WORLD
21 → !
00 → string terminator

So the final string stored in memory is:
"HELLO WORLD!"